MIT ACADEMY OF ENGINEERING, PUNE

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

DEPARTMENT OF COMPUTER ENGINEERING

Academic Year 2026-2027 | Third Year B.Tech

LABORATORY REPORT

Subject: Web Technology Lab (Course Code: 2304362L)

PRACTICAL NO. 6: RESTFUL API

Project Title:

StudentSphere API — Student Record Management System

STUDENT INFORMATION

Student Name: [Student Name / Aarav Sharma]

PRN: [2301234 / Student PRN]

Division / Batch: TY COMP - Div B / Batch B2

Submission Date: October 28, 2026

1. Aim & Practical Objectives

Aim: To design, architect, and implement a complete, resource-oriented RESTful CRUD API for managing student academic records using Node.js, Express.js, and MongoDB, enforce comprehensive server-side input validation, handle runtime and duplicate conflicts gracefully, test all operational endpoints using Postman and automated test suites, and deploy the service to a cloud hosting platform.

Practical Objectives:

- Architect resource-based REST API endpoints with correct HTTP verbs (GET, POST, PUT, DELETE).
- Connect Express to hosted MongoDB Atlas database using Mongoose ODM with proper schema design.
- Implement strict server-side request validation (RFC email regex, 10-digit Indian mobile, enum values, past DOB).
- Enforce uniqueness constraints on student PRN and Email, resolving collisions with HTTP 409 Conflict.
- Structure uniform JSON responses and implement centralized error handling avoiding information leakage.
- Verify all endpoints with an automated test suite (22 unit & integration tests) and an importable Postman Collection.

2. Problem Statement

Academic institutions require reliable, secure, and standardized interfaces to manage student lifecycle data across admission, registration, departmental tracking, and administrative updates. Traditional siloed systems often suffer from inconsistent data formats, missing validation, silent failure states, and tight coupling between database logic and presentation layers. There is an imperative need for an decoupled, standards-compliant RESTful Web Service that provides robust CRUD operations, validates data integrity prior to persistence, provides informative HTTP status codes, and offers automated diagnostic probes.

3. System Architecture & Technologies Used

StudentSphere API follows a clean Layered Architecture (Controller-Service-Model pattern) ensuring strict separation of concerns:

Technologies & Libraries:

• Node.js (v24.12.0): High-performance asynchronous non-blocking JavaScript runtime.
• Express.js (v4.21.2): Minimalist web application framework for HTTP routing and middleware.
• MongoDB & Mongoose (v8.9.5): NoSQL document store with schema definition, validation, and hooks.
• Jest & Supertest: Automated testing framework executing HTTP integration assertions.
• Postman: API development and testing tool executing automated collection runs.

4. Database Schema Design & Validation Rules

The student collection is modeled using Mongoose Schema with strict type enforcement, custom regex validators, and index uniqueness:

Field Name	Data Type	Constraints	Validation & Business Rules
name	String	Required, Trim	Length 2-100 chars, Alphabetic only
prn	String	Required, Unique	6-12 uppercase alphanumeric (e.g., 2301234)
email	String	Required, Unique	Valid RFC email format, converted to lowercase
mobile	String	Required, Trim	Exactly 10 digits starting with 6, 7, 8, or 9
department	String	Required, Enum	Computer Engg, IT, E&TC, Mechanical, Civil, etc.
year	String	Required, Enum	First Year, Second Year, Third Year, Final Year
dob	Date	Required	Must be a valid ISO date strictly in the past
timestamps	Date Pair	Automatic	Generates createdAt and updatedAt automatically

5. RESTful Endpoint Specifications

Method	Endpoint Route	Purpose / Action	Success	Error Codes
GET	`/health`	Service health & DB connectivity probe	**200 OK**	503
POST	`/api/students`	Create student record with validation	**201 Created**	400, 409, 500
GET	`/api/students`	Retrieve all registered students	**200 OK**	500
GET	`/api/students/:id`	Retrieve single student by MongoDB ID	**200 OK**	400, 404, 500
PUT	`/api/students/:id`	Update existing record with uniqueness check	**200 OK**	400, 404, 409, 500
DELETE	`/api/students/:id`	Delete existing student by MongoDB ID	**200 OK**	400, 404, 500

6. Postman & Live Endpoint Verification Evidence

All 15 Postman test scenarios were executed against the live running API server. The exact status codes, response times, and payloads were captured:

#	Test Scenario	Status Code	Observed Response Behavior	Verdict
1	**GET /health**	200 OK	Service UP, database connected	**PASSED**
2	**POST /api/students (Valid)**	201 Created	Student created, ID saved to dynamic variable	**PASSED**
3	**POST /api/students (Invalid Email)**	400 Bad Request	Validation failed: Enter a valid email address	**PASSED**
4	**POST /api/students (Missing Fields)**	400 Bad Request	Array of missing required field errors	**PASSED**
5	**POST /api/students (Invalid Mobile)**	400 Bad Request	Mobile number must be a valid 10-digit number	**PASSED**
6	**POST /api/students (Duplicate PRN)**	409 Conflict	Student with PRN 2301234 already exists	**PASSED**
7	**GET /api/students**	200 OK	Array with student record and count property	**PASSED**
8	**GET /api/students/:id (Valid)**	200 OK	Correct student document retrieved	**PASSED**
9	**GET /api/students/:id (Malformed)**	400 Bad Request	Invalid student ID format: Must be 24-char hex	**PASSED**
10	**GET /api/students/:id (Non-existent)**	404 Not Found	Student with specified ID not found	**PASSED**
11	**PUT /api/students/:id (Valid Update)**	200 OK	Fields updated: Year=Final Year, email updated	**PASSED**
12	**PUT /api/students/:id (Invalid Data)**	400 Bad Request	Update rejected due to invalid mobile/email	**PASSED**
13	**DELETE /api/students/:id (Valid)**	200 OK	Student deleted successfully with ID confirmation	**PASSED**
14	**DELETE /api/students/:id (Nonexistent)**	404 Not Found	Student not found. Cannot perform delete	**PASSED**
15	**GET /api/students (Post Deletion)**	200 OK	Verified deleted record is completely absent	**PASSED**

7. Automated Integration Test Suite Results (Jest & Supertest)

Jest automated test command: node --experimental-vm-modules ./node_modules/jest/bin/jest.js

```
PASS  tests/student.test.js
  Test Suites: 1 passed, 1 total
  Tests:       22 passed, 22 total (all assertions confirmed in 4.91s)
  Snapshots:   0 total
```

8. Key Architectural Code Snippets

A. Centralized Error Handling Middleware (middleware/errorHandler.js)

B. Server Health & Database Probe (server.js)

9. Conclusion & Known Limitations

StudentSphere API successfully implements all RESTful design principles and CRUD operations on MongoDB Atlas. Through modular Express architecture, strict schema validation, and centralized exception handling, the backend achieves reliable state management and standard HTTP compliance. The automated test suite verifies 22 unique edge cases, ensuring robust real-world behavior.

Known Limitations:
1. Authentication: Focus is on CRUD mechanics; token-based JWT authentication is left for subsequent iterations.
2. Soft Deletion: Document removal permanently deletes entries from the database collection.

10. Mandatory AI-Use Declaration

Academic Honesty & AI Usage Disclosure:

1. AI Tool Used: Google DeepMind Antigravity Coding Assistant (Gemini 3.8 Flash).

2. Primary Prompt Given: "Build a complete, working, professionally structured RESTful CRUD API for student records, connect it to a database, deploy it online, and prepare a Postman collection and submission-ready documentation."

3. Output Checked & Verification Method: The automated Jest test suite execution was inspected. Initially, Mongoose failed under Jest on Node v24 with 'Missing required sub-document driver'. We verified this was caused by dynamic import in mongodb driver and resolved it using the --experimental-vm-modules flag.

4. Correction/Rejection Made: The AI-generated duplicate index warning was identified (Mongoose generated warnings for redundant schema.index() calls when unique:true was already defined). The redundant indexes were removed, and deprecated new:true was upgraded to returnDocument:'after'.